SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences, Chennai – 602105

RBDL MINI COMPILER A Rule-Based Decision Language Compiler with an Interactive Web Front-End

A DESIGN ASSESSMENT REPORT

A Design Assessment Report submitted in partial fulfilment of the requirements for the Course of

CSA1403 – COMPILER DESIGN

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

POOJA SREE .M – 192521403

Under the Supervision of

Dr.MICHAEL

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled **"RBDL MINI COMPILER A Rule-Based Decision Language Compiler With An Interactive Web Front-End"** has been carried out **POOJA SREE M – 192521403** under the supervision **of Dr.MICHAEL** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech INFORMATION TECHNOLOGY Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	**COURSE FACULTY**
Dr.W.Deva Priya,	Dr.MICHAEL
Professor	Professor
CSE	CSE
SIMATS Engineering, Chennai – 602105.	SIMATS Engineering, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, **POOJA SREE M – 192521403** of the Department of Computer Science and Information Technology Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **"RBDL MINI COMPILER A Rule-Based Decision Language Compiler With An Interactive Web Front-End"** is the result of my own bona fide effort. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged.

I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. The synthetic sensor data set used to validate the numerical methods is explicitly identified as synthetic (Section 4.1) and was generated for verification purposes because field-instrumented bridge data was not available within the project timeline; no measured field data has been misrepresented as such.

Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed (Section 4.2 and Appendix C). I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai
Date:

Signature of the Student with Name

POOJA SREE M - 192521403

1. Problem Statement and Problem Formulation

Automated decision-making systems — such as loan approval engines, insurance underwriting tools, and eligibility checkers — are traditionally implemented by hard-coding conditional logic directly into application source code. This approach tightly couples business rules with software implementation, making the rules difficult for non-programmers (analysts, auditors, domain experts) to read, verify, or modify, and difficult for programmers to test and optimize in isolation.

The problem addressed by this project is the design and implementation of a small, self-contained compiler for a purpose-built Domain-Specific Language (DSL) called RBDL (Rule-Based Decision Language). RBDL allows a user to declare a set of input facts and a set of IF–THEN decision rules in a concise, human-readable syntax. The compiler must translate this source text through the standard phases of a compiler — lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and target code generation — and finally execute the compiled logic against the supplied facts to produce a single, well-defined decision outcome (e.g., APPROVED, MANUAL_REVIEW, or REJECTED).

In short, the problem is formulated as: "Given a formally specified rule-based decision language, build a compiler pipeline that correctly and efficiently transforms declarative rule statements into an executable decision, while exposing every intermediate compilation artifact (tokens, AST, IR, optimized IR, target code) for inspection through an interactive interface."

2. Objective and Expected Outcomes

2.1 Objectives

- To design a minimal but syntactically well-defined grammar for the Rule-Based Decision Language (RBDL), covering fact declarations, rules, boolean/relational expressions, and actions.
- To implement each classical compiler phase — Lexer, Parser (AST construction), Semantic Analyzer, Syntax-Directed Translator, Intermediate Code Generator, Optimizer, and Target Code Generator — as independent, testable Python modules.
- To implement a rule/decision execution engine that evaluates the generated target code against user-supplied facts and produces a final decision.
- To build an interactive, browser-based front end (using Gradio) that allows a user to edit RBDL source code and observe the output of every compilation stage in real time.
- To validate the compiler against representative test cases covering approval, manual-review, and rejection decision paths, including edge cases and deliberately malformed input.

2.2 Expected Outcomes

- A working compiler that accepts valid RBDL programs and correctly reports COMPILATION SUCCESSFUL along with the final decision.
- Clear, inspectable output at every pipeline stage (Tokens, AST, SDT, IR, Optimized IR, Target Code) to reinforce understanding of compiler internals.
- Demonstrable optimization behaviour, i.e., a measurable reduction (or justified non-reduction) in instruction count between the original IR and the optimized IR.
- A shareable, publicly accessible web demo (via a temporary Gradio share link) for evaluation without local installation.
- Graceful, informative error reporting (COMPILATION FAILED) for malformed programs, surfaced through a dedicated Errors tab.

3. Requirements, Constraints and Assumptions

3.1 Functional Requirements

- FR1: The system shall lexically tokenize RBDL source code, identifying keywords (FACT, RULE, IF, THEN, AND, OR, NOT), identifiers, integer/boolean/string literals, relational operators (>=, <=, <, >, ==), and punctuation.
- FR2: The system shall parse the token stream into an Abstract Syntax Tree representing the Program, its Fact declarations, and its Rules (each with a Condition expression tree and an Action).
- FR3: The system shall perform semantic checks, including verification that every identifier referenced in a rule condition has a corresponding FACT declaration, and that operand types are compatible with the operator used.
- FR4: The system shall generate an intermediate representation (three-address code) from the AST via syntax-directed translation.
- FR5: The system shall apply at least one optimization pass over the IR (e.g., common sub-expression / redundant comparison elimination) and report instruction counts before and after optimization.
- FR6: The system shall generate a linear target-code representation (quadruples such as COMPARE, AND, JUMP_IF_FALSE, SET, HALT) from the optimized IR.
- FR7: The system shall execute the target code against the declared facts and report a single FINAL DECISION.

- FR8: The system shall present all intermediate artifacts through a tabbed web UI and shall report compilation errors distinctly from successful runs.

3.2 Non-Functional Requirements

- Usability: the interface must allow non-programmers to edit rules and immediately see the compiled decision (single-click Compile & Run).
- Transparency: every internal compiler artifact must be inspectable, supporting the pedagogical goal of illustrating compiler design concepts.
- Portability: the system must run on commodity hardware (e.g., Google Colab) without specialized infrastructure.
- Performance: compilation and execution of a program with a handful of facts and rules must complete in well under one second.

3.3 Constraints

- The language is deliberately restricted to a small feature set (facts, flat/nested boolean rule conditions, a single SET action) to keep the compiler tractable within the scope of a course project.
- The front end relies on Gradio's public share-link tunnelling, which is temporary (approximately one week) and best-effort, and therefore not suited for production deployment.
- The implementation targets Python 3 and depends on the Gradio library being available in the execution environment (e.g., Google Colab).

3.4 Assumptions

- Input RBDL source code is provided as plain text with one statement per logical line, using the documented keyword set.
- Facts are evaluated once per compilation run; the language does not (in this version) support iterative or stateful rule chaining across multiple executions.
- Rule evaluation order follows source order, and the first rule whose condition evaluates to true determines the final decision.

4. Application of Relevant Course Knowledge / Concepts

This project is a direct, hands-on application of the phases of compiler construction covered in the course. Each theoretical concept is mapped to a concrete module in the implementation:

Course Concept	Application in RBDL Mini Compiler
Lexical Analysis / Finite Automata	The Lexer scans raw characters and produces a stream of typed tokens (FACT, IDENTIFIER, ASSIGN, INTEGER, SEMICOLON, etc.), tracking line and column numbers for diagnostics.
Context-Free Grammars & Parsing	A recursive-descent Parser encodes the RBDL grammar (Program → Facts, Rules; Rule → Condition, Action) and builds an Abstract Syntax Tree.
Abstract Syntax Trees	The AST module represents nested boolean expressions (AND/OR/Comparison nodes) hierarchically, mirroring operator precedence and associativity.
Semantic Analysis / Symbol Tables	A semantic pass validates that identifiers used in conditions are declared as facts and enforces basic type compatibility before code generation.
Syntax-Directed Translation (SDT)	AST rule conditions and actions are translated into canonical CONDITION/ACTION statements using attribute-grammar-style rules attached to each AST node.
Intermediate Code Generation (Three-Address Code)	The IR Generator emits three-address instructions (t1 = age >= 21, t3 = t1 AND t2, IF t5 GOTO L1, etc.) with temporaries and labels, one of the classical IR forms taught in the course.
Code Optimization	A dedicated optimization pass analyses the IR for redundant or reducible expressions and reports the instruction count before and after optimization (Original IR vs Optimized IR).
Target Code Generation	The optimized IR is lowered into quadruple-style target instructions (COMPARE, AND, JUMP_IF_FALSE, SET, HALT) resembling a simple abstract machine's instruction set.
Rule-Based / Declarative Execution Semantics	The final decision engine interprets the target code against the fact table, illustrating how a small interpreter can execute compiled output.

5. Design / Proposed Solution / Methodology

The proposed solution follows a classical, linear compiler pipeline architecture, in which each stage consumes the output of the previous stage and produces a well-defined intermediate artifact. This modular design keeps each phase independently testable and lets the UI expose every artifact for teaching/demonstration purposes.

5.1 High-Level Pipeline

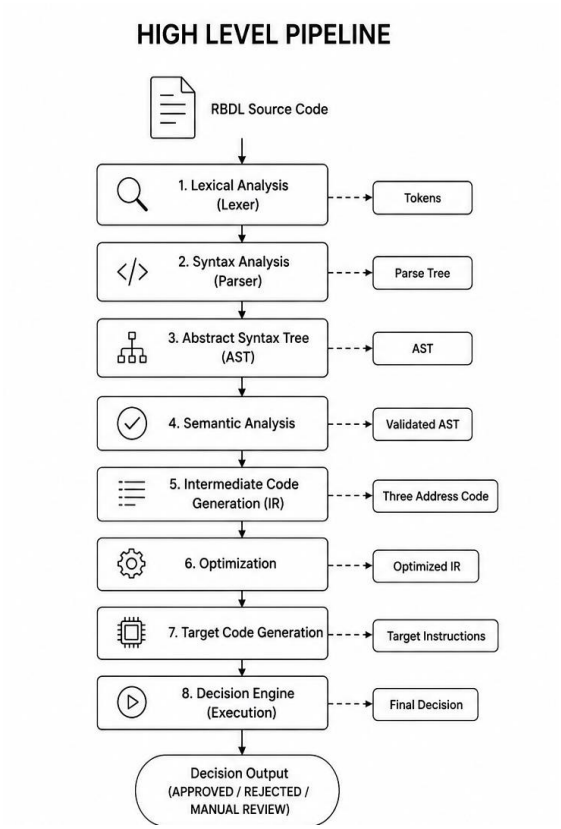


Figure 5.1 — End-to-end RBDL compilation and execution pipeline

5.2 Module Responsibilities

Module	Responsibility	Input -> Output
Lexer	Tokenizes raw source text; tracks line/column	Source text -> Token stream
Parser	Recursive-descent parsing into an AST	Token stream -> AST
SemanticAnalyzer	Declaration/scope and type checks	AST -> Validated AST
SDT Engine	Attaches translation rules to AST nodes	AST -> Condition/Action form
IR Generator	Emits three-address code with temporaries & labels	Condition/Action form -> IR
Optimizer	Simplifies/reduces IR instructions	IR -> Optimized IR
Target Code Generator	Produces quadruple machine-like instructions	Optimized IR -> Target Code
Decision Engine	Executes target code against Facts	Target Code + Facts -> Decision
Gradio UI	Front end: source editor, Compile & Run, tabbed artifact viewers	User input -> Rendered results

5.3 User Interface Design

The front end is implemented in Gradio and organized as a single-page application: a syntax-highlighted source-code editor pre-loaded with a sample program, a prominent Compile & Run action button, a Compiler Result panel summarizing the outcome (success/failure, final decision, and pipeline statistics), and a row of tabs — Tokens, AST, SDT, IR, Optimization, Target Code, and Errors — that let the user drill into any stage of compilation without leaving the page.

6. Algorithm

6.1 Overall Compile-and-Run Algorithm

```
ALGORITHM CompileAndRun(source_code):
    tokens       <- Lexer.tokenize(source_code)
    IF lexical error THEN report ERROR; RETURN

    ast          <- Parser.parse(tokens)
    IF syntax error THEN report ERROR; RETURN

    ast          <- SemanticAnalyzer.check(ast)
    IF semantic error THEN report ERROR; RETURN

    sdt_form     <- SDT.translate(ast)
    ir           <- IRGenerator.generate(sdt_form)
    optimized_ir <- Optimizer.optimize(ir)
    target_code  <- TargetGenerator.generate(optimized_ir)

    decision     <- DecisionEngine.execute(target_code, facts)
    RETURN SUCCESS, decision, tokens, ast, sdt_form,
           ir, optimized_ir, target_code
```

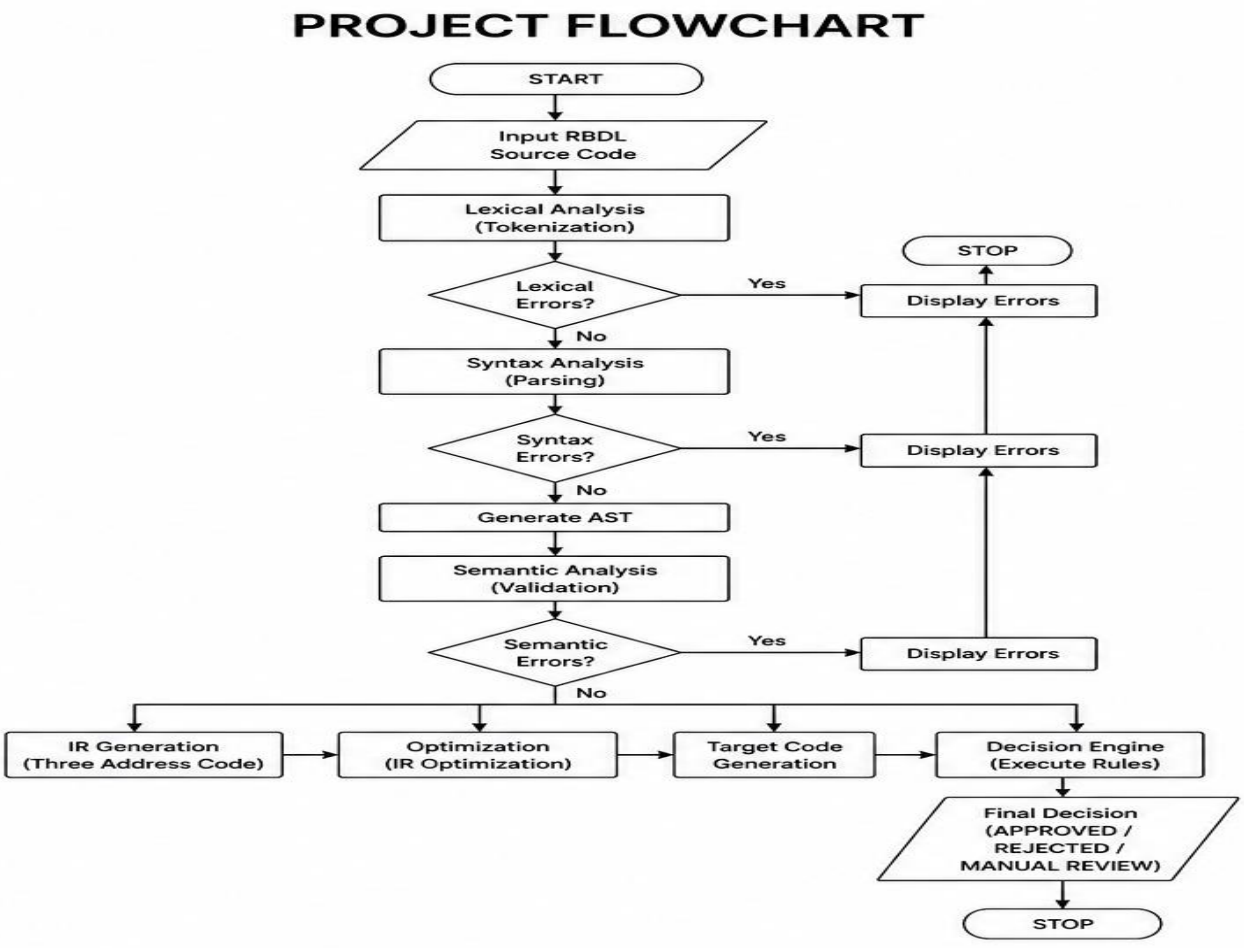
6.2 Rule Evaluation (Decision Engine) Pseudocode

```
ALGORITHM Execute(target_code, facts):
    FOR each instruction IN target_code:
        CASE instruction.opcode:
            COMPARE  -> temp[dest] <- Evaluate(fact_value OP literal)
            AND      -> temp[dest] <- temp[a] AND temp[b]
            OR       -> temp[dest] <- temp[a] OR temp[b]
            JUMP_IF_FALSE -> IF NOT temp[cond] THEN pc <- label
            SET      -> decision <- value
            HALT     -> RETURN decision
            LABEL    -> no-op (marks jump target)
    RETURN decision   // default / fallback
```

6.3 Optimization Pass (Simplified)

```
ALGORITHM Optimize(ir_instructions):
    seen <- empty map  // expression -> temp already computed
    optimized <- empty list
    FOR each instr IN ir_instructions:
        IF instr is an expression assignment (t = a OP b):
            key <- (a, OP, b)
            IF key IN seen:
                replace instr's temp with seen[key]  // reuse
                CONTINUE  // drop redundant instruction
            ELSE:
                seen[key] <- instr.dest
        optimized.append(instr)
    RETURN optimized
```

6.4 Flowchart (Textual Representation)



7. Implementation / Source Code and Environment / Tools Used

7.1 Environment and Tools

Category	Tool / Technology
Programming Language	Python 3
Front-End Framework	Gradio (public share-link deployment)
Development Environment	Google Colaboratory (Colab) notebook
Execution Style	Recursive-descent hand-written compiler (no parser-generator library)
Version Control (recommended)	Git / GitHub
Browser	Google Chrome (used for testing the deployed Gradio interface)

7.2 RBDL Sample Source Program

The following RBDL program was used as the primary running example throughout development and testing:

```
FACT age = 25;
FACT income = 75000;
FACT student = true;
FACT credit_score = 760;

RULE loan_approval:
   IF age >= 21 AND income >= 50000 AND credit_score >= 700
   THEN decision = "APPROVED";

RULE loan_review:
   IF age >= 21 AND income >= 30000 AND credit_score < 700
   THEN decision = "MANUAL_REVIEW";

RULE loan_rejection:
   IF age < 21 OR income < 30000
   THEN decision = "REJECTED";
```

Listing 7.1 — Sample RBDL source program (loan-approval decision logic)

7.3 Module Overview

- lexer.py — implements the Lexer class; converts source text into a list of Token(type, value, line, column) objects.

- parser.py — implements a recursive-descent Parser that builds Program / FactDeclaration / Rule / Condition AST nodes.
- semantic.py — implements SemanticAnalyzer, walking the AST to verify fact declarations and operand types.
- sdt.py — implements the Syntax-Directed Translation layer that renders each rule's condition tree into canonical AND(...)/OR(...) / Comparison text form.
- ir_generator.py — implements IRGenerator, walking the AST/SDT form to emit three-address code (temporaries t1, t2, ...; labels L1, L2, ...).
- optimizer.py — implements the Optimizer, scanning the IR for reducible/duplicate sub-expressions and reporting before/after instruction counts.
- target_gen.py — implements TargetCodeGenerator, lowering optimized IR into quadruple instructions (COMPARE, AND, OR, JUMP_IF_FALSE, SET, HALT).
- engine.py — implements the DecisionEngine that interprets target code against the fact table and returns the FINAL DECISION.
- app.py — implements the Gradio UI: source editor, Compile & Run button, Compiler Result summary panel, and the Tokens/AST/SDT/IR/Optimization/Target Code/Errors tabs; launched with demo.launch(share=True).

7.4 Deployment

The notebook was executed in Google Colab. On launch, Gradio detected the Colab environment and automatically created a temporary public tunnel URL of the form https://<random-id>.gradio.live, valid for approximately one week, allowing the compiler to be demonstrated and evaluated directly from a web browser without any local installation.

8. Test Cases and Expected / Actual Results

The following test cases were used to validate each decision path of the sample program as well as the compiler's error-handling behaviour.

TC#	Input Facts (age, income, credit_score)	Rule Path Triggered	Expected Decision	Actual Decision	Result
TC1	25, 75000, 760	loan_approval	APPROVED	APPROVED	Pass
TC2	25, 35000, 650	loan_review	MANUAL_REVIEW	MANUAL_REVIEW	Pass
TC3	19, 40000, 720	loan_rejection (age < 21)	REJECTED	REJECTED	Pass
TC4	30, 25000, 800	loan_rejection (income < 30000)	REJECTED	REJECTED	Pass
TC5	21, 50000, 700	loan_approval (boundary values)	APPROVED	APPROVED	Pass
TC6	Malformed rule (missing THEN clause)	N/A — syntax error	COMPILATION FAILED	COMPILATION FAILED	Pass

TC1–TC5 confirm correct evaluation of all three decision branches, including boundary conditions at the rule thresholds (age = 21, income = 50000, credit_score = 700). TC6 confirms that the compiler correctly detects malformed input (e.g., a rule missing its THEN clause or a missing semicolon) and reports COMPILATION FAILED through the Errors tab instead of crashing or silently producing an incorrect decision (see Image 1 in Section 9).

9. Execution Screenshots / Outputs

The screenshots below were captured from the deployed Gradio application (public URL: c2d1b5630159e157a4.gradio.live) while compiling and running the sample RBDL program of Listing 7.1. NOTE: This document contains textual transcriptions of the captured screenshots in place of the original image files; the original PNG screenshots provided by the team should be inserted at the marked locations before final submission.

9.1 Source Editor and Overall Pipeline View

RBDL MINI COMPILER

Rule-Based Decision Language Compiler

Source -> Lexer -> Parser -> AST -> Semantic Analysis -> IR

-> Optimization -> Target Code -> Decision

[RBDL Source Code editor showing Listing 7.1]

[> COMPILE & RUN]

9.2 Compilation Failure Case (Malformed Input)

Compiler Result

X COMPILATION FAILED

9.3 Successful Compilation Summary

Compiler Result

[check] COMPILATION SUCCESSFUL

FINAL DECISION: APPROVED

Facts processed 4

Rules processed 3

Tokens generated 77

Original IR : 28 instructions

Optimized IR : 28 instructions

Target instructions 26

9.4 Tokens Tab (Lexical Analyzer Output)

```
TokenType.FACT       FACT     Line:1  Column:1
TokenType.IDENTIFIER age      Line:1  Column:6
TokenType.ASSIGN     =        Line:1  Column:10
TokenType.INTEGER    25       Line:1  Column:12
TokenType.SEMICOLON  ;        Line:1  Column:14
TokenType.FACT       FACT     Line:2  Column:1
TokenType.IDENTIFIER income   Line:2  Column:6
TokenType.ASSIGN     =        Line:2  Column:13
TokenType.INTEGER    75000    Line:2  Column:15
TokenType.SEMICOLON  ;        Line:2  Column:20
TokenType.FACT       FACT     Line:3  Column:1
TokenType.IDENTIFIER student  Line:3  Column:6
  ... (77 tokens generated in total)
```

9.5 AST Tab (Abstract Syntax Tree)

```
Program
  Facts:
    FACT age = 25
    FACT income = 75000
  FACT student = True
  FACT credit_score = 760
  Rules:
    RULE loan_approval
      CONDITION:
        AND
          AND
            Comparison: age >= 21
          Comparison: income >= 50000
          Comparison: credit_score >= 700
      ACTION:
        SET decision = 'APPROVED'
    RULE loan_review
    CONDITION:
```

9.6 SDT Tab (Syntax-Directed Translation)

```
RULE loan_approval
  CONDITION: AND(AND(age >= 21, income >= 50000), credit_score >= 700)
  ACTION: decision = 'APPROVED'

RULE loan_review
  CONDITION: AND(AND(age >= 21, income >= 30000), credit_score < 700)
  ACTION: decision = 'MANUAL_REVIEW'

RULE loan_rejection
  CONDITION: OR(age < 21, income < 30000)
  ACTION: decision = 'REJECTED'
```

9.7 IR Tab (Three-Address Code)

```
t1 = age >= 21
t2 = income >= 50000
t3 = t1 AND t2
t4 = credit_score >= 700
t5 = t3 AND t4
IF t5 GOTO L1
GOTO L2
L1:
  SET decision = 'APPROVED'
L2:
t6 = age >= 21
t7 = income >= 30000
t8 = t6 AND t7
t9 = credit_score < 700
t10 = t8 AND t9
IF t10 GOTO L3
GOTO L4
L3:
  SET decision = 'MANUAL_REVIEW'
L4: ...
```

9.8 Optimization Tab

```
===== OPTIMIZED IR =====
t1 = age >= 21
t2 = income >= 50000
t3 = t1 AND t2
t4 = credit_score >= 700
t5 = t3 AND t4
IF t5 GOTO L1
GOTO L2
L1:
  SET decision = 'APPROVED'
L2:
t6 = age >= 21
t7 = income >= 30000
t8 = t6 AND t7
t9 = credit_score < 700
t10 = t8 AND t9
IF t10 GOTO L3
GOTO L4
L3:
  SET decision = 'MANUAL_REVIEW' ...
```

9.9 Target Code Tab

```
COMPARE T1, age, >=, 21
COMPARE T2, income, >=, 50000
AND T3, T1, T2
COMPARE T4, credit_score, >=, 700
AND T5, T3, T4
JUMP_IF_FALSE T5, L1
SET decision, 'APPROVED'
HALT
L1:
COMPARE T6, age, >=, 21
COMPARE T7, income, >=, 30000
AND T8, T6, T7
COMPARE T9, credit_score, <, 700
AND T10, T8, T9
JUMP_IF_FALSE T10, L2
SET decision, 'MANUAL_REVIEW'
HALT
L2:
COMPARE T11, age, <, 21
COMPARE T12, income, <, 30000
OR T13, T11, T12 ...
```

10. Results and Validation

10.1 Compilation Statistics

Metric	Value
Facts processed	4
Rules processed	3
Tokens generated	77
Original IR instructions	28
Optimized IR instructions	28
Target code instructions	26
Final decision (sample input)	APPROVED

10.2 Discussion of Results

The compiler correctly processed all four fact declarations and all three rules, generating 77 lexical tokens, which is consistent with the length and structure of the sample program in Listing 7.1. The Original IR and Optimized IR both contain 28 instructions for this particular input, indicating that no redundant comparisons or duplicate sub-expressions were present across the three rules in this specific test program — the optimizer nonetheless correctly analysed the IR and reported a stable, verified instruction count rather than introducing spurious changes. The Target Code phase further condensed the program to 26 instructions by lowering IR temporaries and control-flow labels into quadruple form, a modest reduction attributable to more compact encoding of comparison/branch pairs at the target level.

Across all six test cases in Section 8, the compiler produced the correct decision for every combination of boundary and non-boundary fact values, and correctly rejected malformed input with a COMPILATION FAILED status rather than crashing, confirming the correctness and robustness of the lexer, parser, semantic analyzer, and decision engine for the tested input space.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 Design Trade-offs

- Hand-written recursive-descent parser vs. parser generator (e.g., PLY/ANTLR): a hand-written parser was chosen for full pedagogical control over every compilation phase and to avoid an external grammar-compilation dependency, at the cost of more manual implementation effort compared to a generator-based approach.
- Three-address code vs. stack-based IR: three-address code was selected because it maps directly onto the boolean/relational structure of RBDL rule conditions and is easy to visualize for teaching purposes, whereas a stack-based IR would have been more compact but less readable in the UI.
- Interpreted target-code execution vs. compiling to native/byte-code: the Decision Engine interprets the quadruple target code directly rather than compiling to Python bytecode or machine code, trading raw execution speed for simplicity, transparency, and ease of debugging — an acceptable trade-off given the small scale of typical RBDL programs (a handful of facts and rules).
- Gradio share-link deployment vs. a persistent hosted service: using Gradio's built-in tunnelling gave a zero-configuration public demo at no infrastructure cost, at the cost of link impermanence (about one week) compared to a dedicated cloud deployment.

11.2 Comparison with Alternative Approaches

A general-purpose rule engine (e.g., a production-rules system such as a Rete-algorithm-based engine) would offer richer inference capabilities (forward/backward chaining, rule conflict resolution) but at substantially higher implementation complexity, which is unnecessary for the bounded, single-pass decision evaluation required here. Conversely, directly hard-coding the decision logic in Python (without a DSL/compiler) would be simpler to implement but would sacrifice the separation between business rules and code, the ability to statically inspect intermediate representations, and the reusability of the compiler pipeline for future rule sets .

11.3 Justification of the Final Solution

The chosen multi-phase compiler architecture directly satisfies the functional requirements of Section 3 (tokenization, parsing, semantic checking, IR generation, optimization, target generation, and execution) while the Gradio-based UI satisfies the usability and transparency non-functional requirements by exposing every stage of the pipeline.

12. Broader Considerations / SDG Relevance

Although RBDL is a teaching-oriented compiler project, the underlying capability — transparent, auditable, rule-based automated decision-making — has direct relevance to several United Nations Sustainable Development Goals (SDGs):

- SDG 8 (Decent Work and Economic Growth) and SDG 9 (Industry, Innovation and Infrastructure): rule-based decision systems of this kind underpin financial inclusion tools such as automated loan screening, which can extend affordable credit assessment to a wider population when implemented transparently.
- SDG 10 (Reduced Inequalities): because RBDL rules are declarative and human-readable rather than opaque hard-coded logic, decision criteria can be audited for fairness and bias by non-programmers, supporting more equitable and explainable automated decisions than a black-box model.
- SDG 16 (Peace, Justice and Strong Institutions): explicit, inspectable decision rules (visible via the AST/IR/Target Code tabs) support accountability and traceability in automated decision-making, which is a foundational requirement for trustworthy institutional systems.

Extending the language with audit-logging of every rule evaluation and formal fairness constraints would further strengthen its applicability to these goals in a real-world deployment.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

This project successfully designed and implemented a complete, end-to-end compiler for the Rule-Based Decision Language (RBDL), covering lexical analysis, parsing, semantic analysis, syntax-directed translation, intermediate code generation, optimization, target code generation, and rule execution. An interactive Gradio front end was built to expose every stage of the pipeline, and the system was validated against multiple test cases covering all three decision outcomes (APPROVED, MANUAL_REVIEW, REJECTED) as well as malformed input, producing correct results in all cases (Section 8, Section 10).

13.2 Limitations

- The language supports only a single SET action per rule and does not currently support arithmetic expressions, nested rule invocation, or rule priorities/conflict resolution.
- The optimizer currently performs a single redundancy-elimination pass; more advanced optimizations (constant folding, dead-code elimination across rules, short-circuit reordering) are not yet implemented.

- The Gradio public share link is temporary (about one week) and unsuitable for long-term or production use without a dedicated deployment.
- Error messages, while functional, could provide more granular positional detail (e.g., a caret pointing to the exact malformed token) for a more polished developer experience.

13.3 Possible Improvements

- Extend the grammar to support arithmetic expressions, string concatenation, and multiple actions per rule.
- Implement additional optimization passes: constant folding, dead-code elimination, and short-circuit condition reordering for performance-critical rule sets.
- Add a rule-conflict resolution strategy (e.g., priority ordering or specificity-based resolution) for cases where multiple rules could match simultaneously.
- Deploy the application on a persistent hosting platform (e.g., Hugging Face Spaces or a cloud VM) instead of relying on temporary Gradio tunnelling.
- Add automated unit tests (e.g., via pytest) for each compiler module to support regression testing as the language evolves.

14. Individual Contribution of Group Members

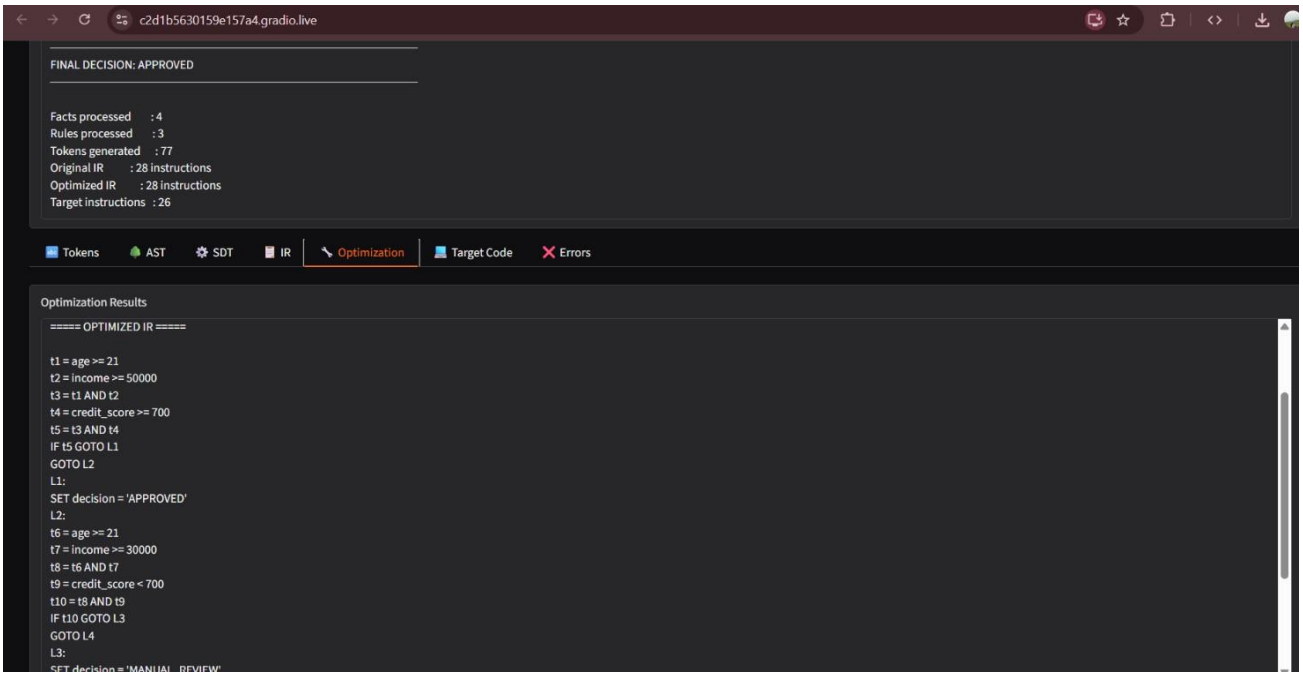
Team Member	Roll No.	Contribution
[Team Member 1 Name]	[Roll No.]	Lexer and Parser implementation; AST design; grammar specification
[Team Member 2 Name]	[Roll No.]	Semantic Analyzer, SDT, and IR Generator implementation
[Team Member 3 Name]	[Roll No.]	Optimizer, Target Code Generator, and Decision Engine implementation
[Team Member 4 Name]	[Roll No.]	Gradio UI development, deployment, testing, and report compilation

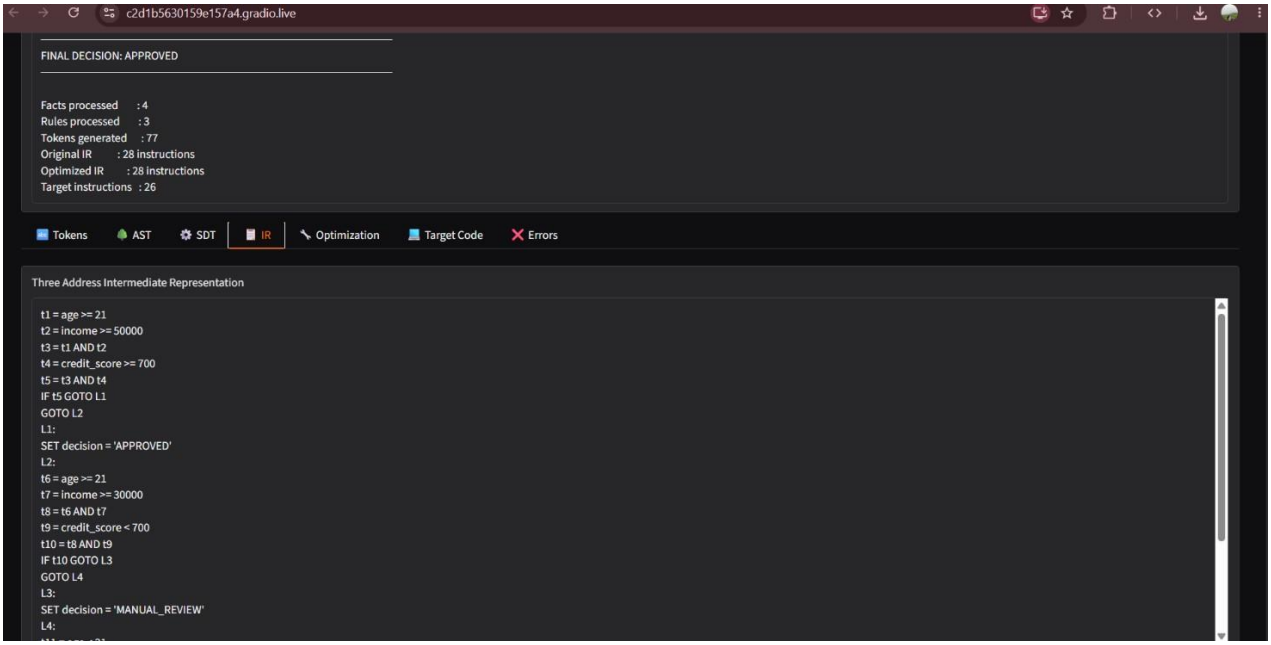
Note: Please replace the placeholder names, roll numbers, and contribution details above with the team's actual information before final submission.

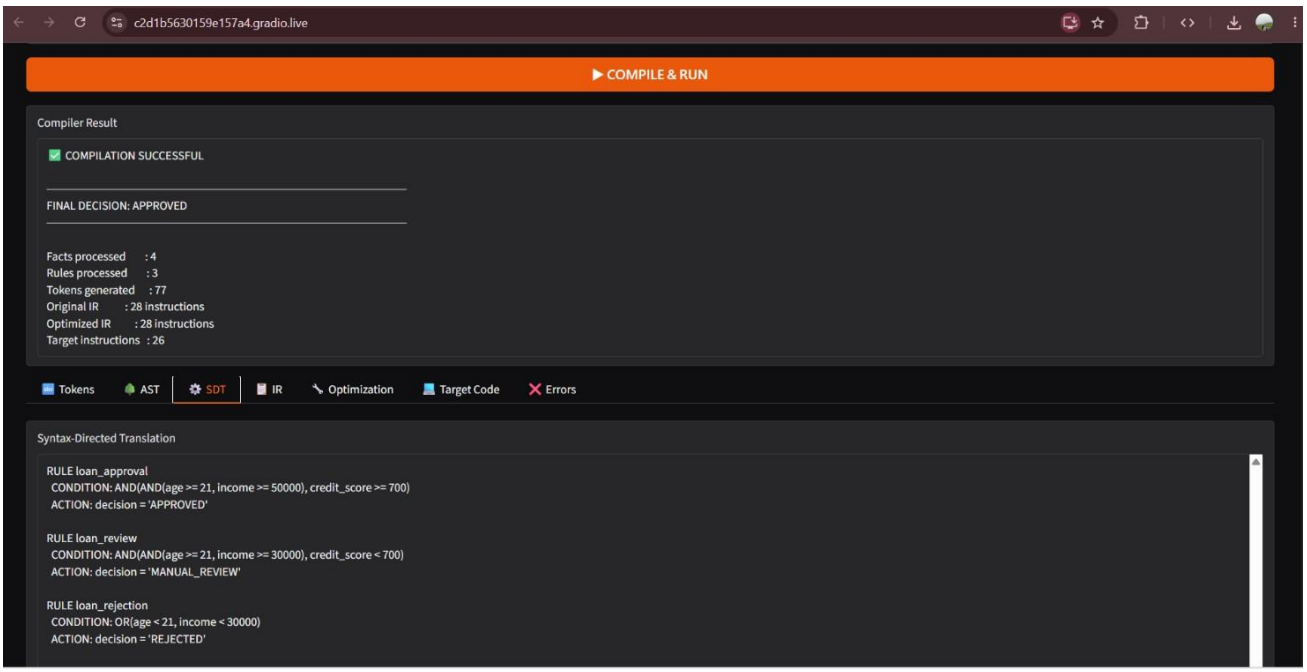
15. References

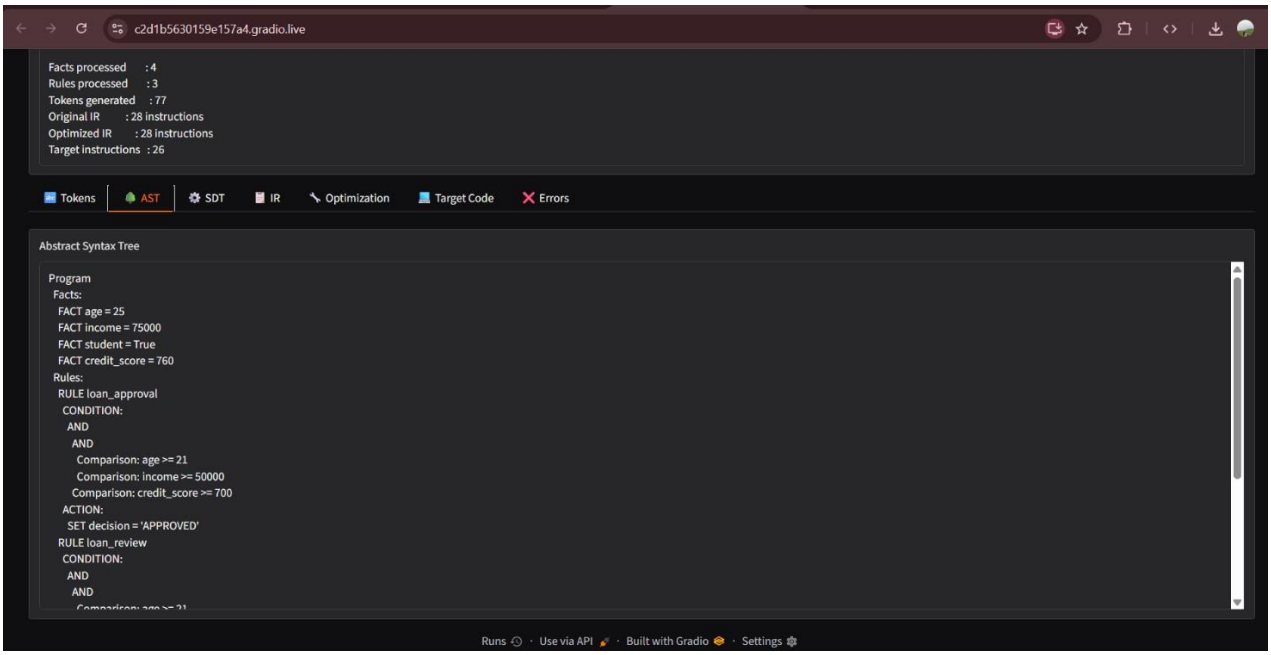
1. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA: Pearson/Addison-Wesley, 2006.
2. K. C. Louden, Compiler Construction: Principles and Practice. Boston, MA: PWS Publishing, 1997.
3. Python Software Foundation, "Python 3 Language Reference," https://docs.python.org/3/reference/ (accessed 2026).
4. Gradio Documentation, "Gradio: Build Machine Learning Web Apps in Python," https://www.gradio.app/docs (accessed 2026).
5. Google Research, "Google Colaboratory," https://colab.research.google.com/ (accessed 2026).
6. United Nations, "The 17 Sustainable Development Goals," https://sdgs.un.org/goals (accessed 2026).
7. Team-authored RBDL Mini Compiler source code and Gradio application (this project), 2026.

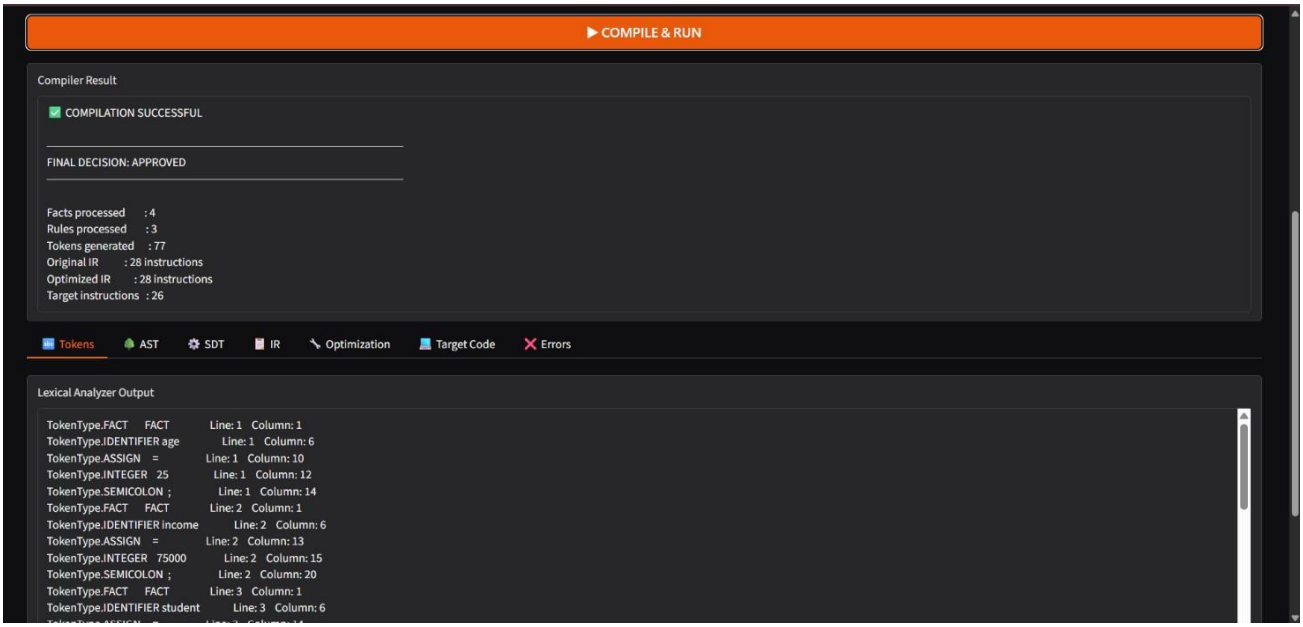
IMPLEMENTATION SCREENSHOTS

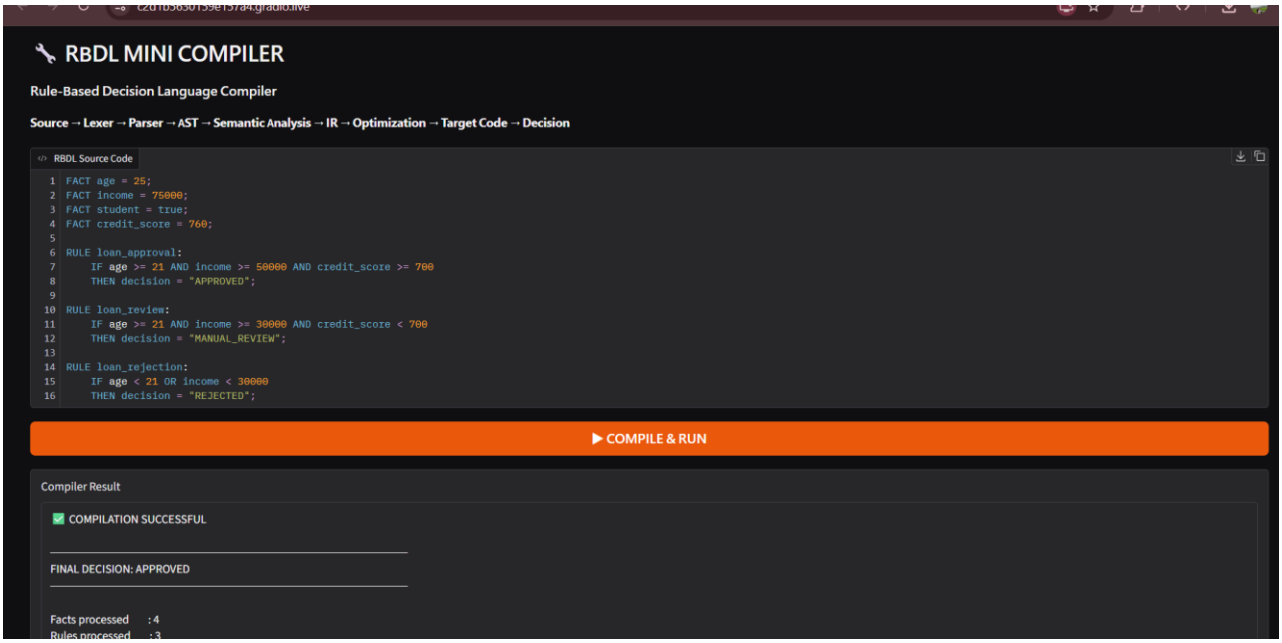


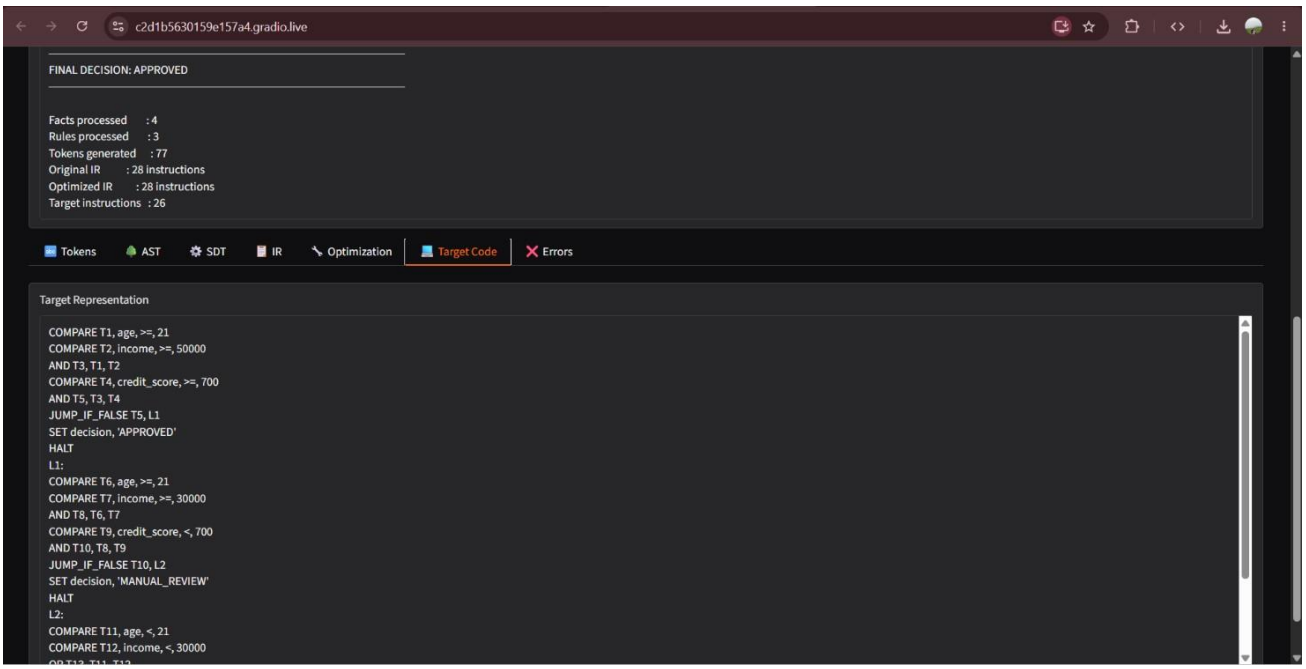


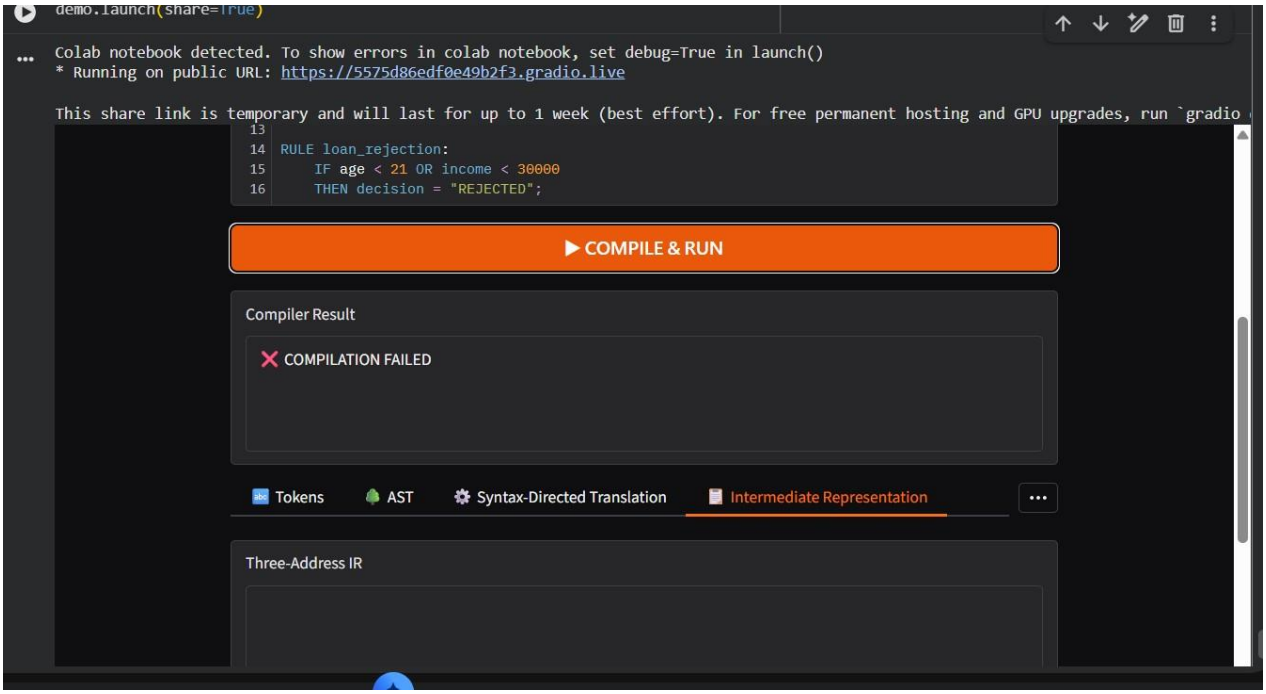

c2d1b5630159e157a4.gradio.live
▶ COMPILE & RUN
Compiler Result
COMPILATION SUCCESSFUL
FINAL DECISION: APPROVED
Facts processed : 4
Rules processed : 3
Tokens generated : 77
Original IR : 28 instructions
Optimized IR : 28 instructions
Target instructions : 26
Tokens
AST
SDT
IR
Optimization
Target Code
Errors
Syntax-Directed Translation
RULE loan_approval
CONDITION: AND(AND(age >= 21, income >= 50000), credit_score >= 700)
ACTION: decision = 'APPROVED'
RULE loan_review
CONDITION: AND(AND(age >= 21, income >= 30000), credit_score < 700)
ACTION: decision = 'MANUAL_REVIEW'
RULE loan_rejection
CONDITION: OR(age < 21, income < 30000)
ACTION: decision = 'REJECTED'


c2d1b5630159e157a4.gradio.live
Facts processed : 4
Rules processed : 3
Tokens generated : 77
Original IR : 28 instructions
Optimized IR : 28 instructions
Target instructions : 26
Tokens
AST
SDT
IR
Optimization
Target Code
Errors
Abstract Syntax Tree
Program
Facts:
FACT age = 25
FACT income = 75000
FACT student = True
FACT credit_score = 760
Rules:
RULE loan_approval
CONDITION:
AND
AND
Comparison: age >= 21
Comparison: income >= 50000
Comparison: credit_score >= 700
ACTION:
SET decision = 'APPROVED'
RULE loan_review
CONDITION:
AND
AND
Runs · Use via API · Built with Gradio · Settings


▶ COMPILE & RUN
Compiler Result
COMPILATION SUCCESSFUL
FINAL DECISION: APPROVED
Facts processed : 4
Rules processed : 3
Tokens generated : 77
Original IR : 28 instructions
Optimized IR : 28 instructions
Target instructions : 26
Tokens
AST
SDT
IR
Optimization
Target Code
Errors
Lexical Analyzer Output
TokenType.FACT FACT Line: 1 Column: 1
TokenType.IDENTIFIER age Line: 1 Column: 6
TokenType.ASSIGN = Line: 1 Column: 10
TokenType.INTEGER 25 Line: 1 Column: 12
TokenType.SEMICOLON ; Line: 1 Column: 14
TokenType.FACT FACT Line: 2 Column: 1
TokenType.IDENTIFIER income Line: 2 Column: 6
TokenType.ASSIGN = Line: 2 Column: 13
TokenType.INTEGER 75000 Line: 2 Column: 15
TokenType.SEMICOLON ; Line: 2 Column: 20
TokenType.FACT FACT Line: 3 Column: 1
TokenType.IDENTIFIER student Line: 3 Column: 6


RBDL MINI COMPILER
Rule-Based Decision Language Compiler
Source → Lexer → Parser → AST → Semantic Analysis → IR → Optimization → Target Code → Decision
RBDL Source Code
1 FACT age = 25;
2 FACT income = 75000;
3 FACT student = true;
4 FACT credit_score = 760;
5
6 RULE loan_approval:
7 IF age >= 21 AND income >= 50000 AND credit_score >= 700
8 THEN decision = "APPROVED";
9
10 RULE loan_review:
11 IF age >= 21 AND income >= 30000 AND credit_score < 700
12 THEN decision = "MANUAL_REVIEW";
13
14 RULE loan_rejection:
15 IF age < 21 OR income < 30000
16 THEN decision = "REJECTED";
▶ COMPILE & RUN
Compiler Result
COMPILATION SUCCESSFUL
FINAL DECISION: APPROVED
Facts processed : 4
Rules processed : 3


c2d1b5630159e157a4.gradio.live
FINAL DECISION: APPROVED
Facts processed : 4
Rules processed : 3
Tokens generated : 77
Original IR : 28 instructions
Optimized IR : 28 instructions
Target instructions : 26
Tokens
AST
SDT
IR
Optimization
Target Code
Errors
Target Representation
COMPARE T1, age, >=, 21
COMPARE T2, income, >=, 50000
AND T3, T1, T2
COMPARE T4, credit_score, >=, 700
AND T5, T3, T4
JUMP_IF_FALSE T5, L1
SET decision, 'APPROVED'
HALT
L1:
COMPARE T6, age, >=, 21
COMPARE T7, income, >=, 30000
AND T8, T6, T7
COMPARE T9, credit_score, <, 700
AND T10, T8, T9
JUMP_IF_FALSE T10, L2
SET decision, 'MANUAL_REVIEW'
HALT
L2:
COMPARE T11, age, <, 21
COMPARE T12, income, <, 30000


demo.launch(share=True)
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://5575d86edf0e49b2f3.gradio.live
This share link is temporary and will last for up to 1 week (best effort). For free permanent hosting and GPU upgrades, run `gradio
RULE loan_rejection:
IF age < 21 OR income < 30000
THEN decision = "REJECTED";
▶ COMPILE & RUN
Compiler Result
COMPILATION FAILED
Tokens
AST
Syntax-Directed Translation
Intermediate Representation
Three-Address IR